大肥鱼

全量工程规范书

版本 1.1 | 2026-11-10 | 生效

唯一权威规范 · 新功能先读这份 · 删除接口先跑调用图

版本 1.1 | **日期** 2026-11-10 | **状态** 生效

适用范围：D:\DSH\plugins 下的桥接内核、全部 feature 插件、测试、脚本、文档，以及它们的部署与回滚。

效力：本规范书是工程标准的唯一权威入口。docs/ENGINEERING.md 只是快速索引；历史文档（ARCHITECTURE / STATUS / HANDOFF）不代表现状。

文档控制

项目	内容
文档名称	大肥鱼全量工程规范书
版本	1.1
生效日期	2026-11-10
适用对象	大肥鱼、维护者、新功能作者、代码审查者
事实源	本规范书正文；易变数字以 JSON / CI 输出为准
解释顺序	本规范书 > 快速索引 > 操作细节 > 历史资料

阅读指南

- **新增 feature**：读第 4、9、13、14 章，照着模板做。
- **修改 bridge**：读第 5、6、9 章，先跑调用图，再动公开 API。
- **部署/回滚**：读第 10 章，使用脚本，不手工猜。
- **代码审查**：直接用第 14 章检查清单。
- **排障**：读第 7、10 章与附录 F 的事故复盘。

目录

第一部分　规范正文

- 0. 唯一权威与阅读顺序
- 1. 工程范围与事实源

- 2. 分层与依赖规则
- 3. 目录与命名标准
- 4. Feature 插件标准
- 5. Bridge 内核标准
- 6. 双向调用合同
- 7. 数据、安全与观测
- 8. 测试与质量门禁
- 9. 变更流程
- 10. 部署、回滚与健康
- 11. 文档标准
- 12. 现有例外与债务
- 13. 快速模板
- 14. 最终检查清单

第二部分　附录

- A. 当前工程基线
- B. Feature 与 Hook 清单
- C. 命令速查
- D. 新 Feature 实操模板
- E. 代码审查清单
- F. 事故复盘与防线
- G. 术语表
- H. 修订记录

规范不是建议

本规范里的“必须”是硬约束；“建议”是默认做法，偏离要在变更说明里写原因。

违反硬约束的改动，即使“能跑”，也不合并。

角色与职责

角色	职责
主人 / 产品负责人	决定功能优先级、是否接受例外、是否回滚
大肥鱼（实现者）	按本规范实现、自测、记录自检、提交变更说明
审查者	检查分层、接口合同、数据安全、测试与回滚
运维者	部署、健康检查、观察日志、执行回滚

同一个人可以兼任多个角色，但**不能省掉对应检查。**

开发流程总览

```
需求 / 问题
  → 判断是否真的需要新能力
  → 设计（数据、接口、权限、失败路径）
  → 实现（feature 优先，bridge 只接线）
  → 单测 / 合同测试 / 手工链路
  → CI + health
  → 快照
  → 重启 / 部署
  → 观察 + 自检记录
  → 必要时回滚
```

变更分级

级别	例子	最低要求
L1 小修	文案、注释、单文件小 bug	语法检查 + 相关测试
L2 功能	新工具、新命令、新 feature	单测 + audit + test:new + CI + 重启
L3 内核	bridge 公开 API、启动、权限、部署	调用图 + 合同测试 + 全量 CI + 快照 + 回滚演练

第一部分　规范正文

0. 唯一权威与阅读顺序

本文件是唯一权威规范。 新增功能只需要读本文件；下面这些是可选参考，不是第二套规则：

文档	定位
docs/README.md	文档索引
docs/ENGINEERING.md	一页速查
docs/TESTING.md	测试细节
docs/DEPLOY.md	部署细节
features/README.md	插件速查；规则以本文件为准
docs/ARCHITECTURE.md / STATUS.md / HANDOFF.md	历史或冻结资料，不作为现状

现有 hook 以 features/status.json 与 node scripts/audit-plugins.mjs 的输出为准；不要靠记忆或旧文档。

---

1. 工程范围与事实源

1.1 源码目录

```
D:\DSH\plugins\
  dsh-qq-onebot-bridge\     桥接内核（本地插件包）
    lib\                    内核与 service
    test\                   单元 / 协议 / 契约 / 部署 smoke
    docs\                   内核设计注记
    package.json            脚本、导出、依赖
  features\                 全部功能插件
    <name>\
      package.json          插件声明
      index.js              hooks 导出
      ...                   工具实现
  node_modules\             junction，依赖解析
```

1.2 运行目录

```
D:\DSH\qqbot\
  data\          运行数据（记忆、权限、归档、文件）
  scripts\       CI / 部署 / 健康检查 / 插件审计
  deploy\        NapCat 与服务部署
  docs\          全工程文档
  backups\       快照与回滚包
  archive\       历史归档（死代码/临时产物，不直接删）
  logs\          运行日志
  tools\         本地工具链
```

1.3 边界

- **源码只放 `plugins`**，运行产物只放 qqbot。
- 禁止在源码目录里放 .bak、日志、临时文件、备份 zip。
- features\status.json 是生成文件，可以保留；disabled.json 按需创建。
- 任何“为了省事”直接把文件复制成 xxx.bak 的做法都不允许；备份放 D:\DSH\qqbot\backups。
- 死代码/废弃模块不直接删：移到 D:\DSH\qqbot\archive\source-legacy-YYYYMMDD，并在归档目录写 README.md 说明原因与恢复方式。
- 临时目录、旧浏览器 profile、历史 .tmp 不留在活跃目录；移入 archive\legacy-active-YYYYMMDD 或按保留策略清理。

---

2. 分层与依赖规则

```
QQ / NapCat / DSH Web
        |
        ▼
  bridge 内核（传输、会话、权限、注入、队列、feature 宿主）
        |  hooks / bridge API
        ▼
  features/<name>（领域功能、工具、命令、事件）
        |  SDK 纯函数 / 注入 bridge 能力
        ▼
  lib/services（可复用服务）
        |
        ▼
  数据目录 / 外部能力（TTS、SVC、MAA、抖音）
```

2.1 允许的依赖

从	允许依赖
feature	node:* 内置模块、本插件目录内文件、@zfqxuz/dsh-qq-onebot-bridge/lib/sdk/index.js
feature	注入的 bridge / defineTool / agentCtx / route / key
bridge 内核	lib/services、lib/core、宿主 DSH 包
service	lib/core、node:*、必要的第三方包
test	任意被测模块（通过依赖注入，不碰真实 QQ）

2.2 禁止的依赖

- feature → lib/bridge.js、lib/onebot.js、lib/* 内核内部文件
- feature → 另一个 feature
- feature → 内核私有字段/函数
- service → feature
- 为了“过编译”复制一份实现到 feature
- 在 bridge.js 里新增业务分支

2.3 插件加载与隔离

- feature-loader 扫描 features/*，跳过 disabled.json
- 每个插件独立 try/catch，一个失败不影响其他
- hook 名全局唯一；重复 hook 会在 status.json 的 duplicates 里出现
- 停用即隔离：目录名写入 features/disabled.json，重启后不加载
- 加载结果写入 features/status.json，用于审计与排障

3. 目录与命名标准

3.1 feature 目录

```
features/<name>/
  package.json      必须
  index.js          必须
  README.md         推荐（说明功能、数据、权限、测试）
  tools/            可选（多工具拆分）
  *.js              可选（一个领域一个文件）
```

命名：

- 目录名：小写、短横线或单词，例如 draw、contacts
- 包名：@fish/feature-<name>
- 工具名：全局唯一，建议 qq_ 前缀
- hook 名：动词开头，例如 registerXxxTools、handleXxx
- 数据文件：<domain>-<purpose>.json，放 D:\DSH\qqbot\data

3.2 文件规模

- 单文件超过约 **500 行**，必须评估拆分
- 一个文件混多个领域，拆成多个文件或插件
- 纯逻辑抽到 lib/services，工具/命令留在插件
- 超过 500 行的遗留文件登记在“现有债务”，不新增同类

3.3 文档

- 全工程标准：docs/ENGINEERING-STANDARD.md（本文件）
- 快速索引：docs/ENGINEERING.md
- 测试门禁：docs/TESTING.md
- 架构说明：docs/ARCHITECTURE-OVERVIEW.md
- 架构决策：docs/adr/NNNN-*.md
- 插件说明：features/README.md
- 每次运行改动：D:\DSH\qqbot\小鲸鱼自检记.txt

4. Feature 插件标准

4.1 必填文件

```
package.json
index.js
```

package.json 模板：

```
{
  "name": "@fish/feature-example",
  "version": "1.0.0",
  "description": "一句话说明这个 feature 做什么",
  "type": "module",
  "main": "index.js",
  "dshFeature": {
    "hooks": ["registerExampleTools"],
    "dataPaths": ["D:\\DSH\\qqbot\\data\\example.json"]
  }
}
```

硬性要求：

1. type 必须是 module
2. dshFeature.hooks 必须与 index.js 导出的 hooks 完全一致
3. 用到的持久化路径必须写进 dataPaths
4. 没有数据文件也要写 dataPaths: []
5. 包名统一 @fish/feature-<name>

4.2 `index.js` 形状

```
import { registerExampleTools } from './example-tools.js'

export const hooks = {
  registerExampleTools,
}
```

通用工具入口（没有领域 hook）：

```
export function registerFeature({ defineTool, agentCtx, route, key, bridge }) {
  // 注册工具
}

export const hooks = { registerFeature }
```

规则：

- 只导出 hooks 与必要的命名导出
- 不在模块顶层执行有副作用的网络/文件操作
- 不在加载阶段抛错；加载失败必须被 feature-loader 捕获

4.3 SDK 规则

feature 只能从以下路径导入桥接能力：

```
@zfqxuz/dsh-qq-onebot-bridge/lib/sdk/index.js
```

当前 SDK 导出：

- 消息与文件：parseMessage / toOneBotFileSource / formatFileSize / formatGroupFiles / formatGroupFolders / matchFileName / sanitizeDownloadName
- 语音：synthesizeTts
- 群运营：ops.js 的规划/格式化纯函数（planGroupSign、planKickMembers、parseOpsArgs 等）
- 互动：engage.js 的 planPoke、parseEngageArgs、formatReactionBoard 等
- 群成员/验证/历史/撤回：members.js、verify.js、history.js、recall.js 的纯函数

如果缺能力：

1. 先判断能否用注入的 bridge 完成
2. 不能则把**纯函数**加到 lib/services 或 lib/core
3. 通过 lib/sdk/index.js re-export
4. 补单元测试
5. 不允许 feature 直接 import 内核实现

4.4 权限与工具

- 工具默认只给主人
- 需要放行其他身份，必须在 lib/tool-surface.js 显式声明，不能靠工具内部绕过
- 写操作（发消息、发文件、改群、踢人、审批）必须过 ActionGate / bridge.writeGate
- 命令参数一律空白分隔；不要接受粘连参数
- 每个关闭分支必须给出真实原因，不能静默失败

4.5 数据处理

- 持久化路径必须来自 dataPaths，不能散落硬编码
- 写文件用“临时文件 + rename”或现有 store，避免半截文件
- JSON 数据必须有 schemaVersion 或兼容策略
- 敏感信息（token、cookie、密码）不得写进源码、日志、README
- 读取外部输入必须限制大小、类型、路径（禁止 .. 逃逸）
- 删除/清理必须可恢复（移入 trash 或备份），不能直接销毁

4.6 错误与用户可见文案

- 面向用户的失败：{ ok: false, detail: '...' } 或 safeReply
- 禁止把原始 JSON、堆栈、内部路径直接贴到群里
- 内部错误用 debugLog 记录，带上下文
- 不要用 catch {} 吞掉影响主流程的错误；至少记录
- 异步任务必须有超时/上限，不能无限等待

4.7 新 feature 的 Definition of Done

☐ 目录、package.json、index.js 齐全

☐ hooks 与 package 声明一致

☐ 只依赖 SDK / 注入能力

☐ 工具名全局唯一

☐ 权限与写操作已过 gate

☐ 数据路径已声明

☐ 有单元测试或可回放测试

☐ node scripts/audit-plugins.mjs 输出 issues=0

☐ npm run test:new 通过

☐ scripts/ci.ps1 通过

☐ 重启后 [feature] loaded <name>

☐ 真实链路至少手测一条

☐ 更新 小鲸鱼自检记.txt

---

5. Bridge 内核标准

5.1 职责

bridge.js 只做：

- 组装依赖
- 暴露公开 API
- 调用 feature hook
- 把传输/会话/注入/权限粘起来

bridge.js 不做：

- 领域业务分支
- 具体命令实现
- 生图/唱歌/抖音/TTS 细节
- 数据解析与格式化（放 services）

5.2 新功能与 bridge

默认：**新功能不改 `bridge.js`。**

只有以下情况允许加最小接线：

1. 需要一个新的 feature hook
2. 需要暴露一个新的公开 bridge 能力给 feature
3. 需要把已有 service 接到启动/会话流程

接线要求：

- 只加“调用/适配”，不加业务分支
- 业务逻辑放 lib/services
- 新增公开方法必须补 bridge-api-contract 测试
- 新增 hook 必须同步 features/README.md 的 hook 表
- 改动后跑 test:new、tsc、audit、重启

5.3 公开 API 合同

bridge.* 是 feature 的公开合同，包括：

- 公开方法
- 公开状态字段（如 config、sessions、trace）
- 通过 bridgeApi 代理暴露的能力

规则：

1. 删除/重命名公开方法前，必须证明调用图为 0
2. 调用图要包含：features/**、message-pipeline.js、reply-router.js、startup-runtime、适配器 service
3. 至少保留一个版本的兼容别名
4. 每次删除/改名必须跑 test/bridge-api-contract.test.mjs
5. 不允许用“文件内没搜到”判断方法没人用

5.4 当前桥接门禁

以文件为准：

```
bridge-size-budget.json         行数预算，只降不升
typecheck-bridge-baseline.json 类型错误基线，只降不升
test/bridge-size.test.mjs       行数守卫
test/bridge-typecheck.test.mjs 类型守卫
```

```
test/bridge-private-fields.test.mjs 私有字段守卫
test/bridge-api-contract.test.mjs   下游调用合同守卫
```

当前值（会随修复变化，以 JSON 为准）：

```
bridge.js 1,119 非空行（预算 1,119）
bridge tsc 基线 0
```

行数口径统一为 **非空行**；scripts/check-bridge-size.mjs 与 test/bridge-size.test.mjs 必须使用同一口径，禁止一个算总行、一个算非空行。

行数不是第一目标。 正确顺序是：

```
接口稳定 → 调用迁移 → 测试通过 → 行数自然下降
```

为了压行删除仍被下游调用的方法，属于事故，不是优化。

---

6. 双向调用合同

6.1 bridge → feature

调用点统一模式：

```
const feature = __FEATURES.get('hookName')
if (!feature) { debugLog('[feature] hookName 未加载，功能已停用'); return }
return feature(ctx)
```

要求：

- strict：没有插件就不注册工具，不回退
- 不把 feature 不存在当成异常抛给用户
- 新 hook 必须登记到 features/README.md
- audit-plugins.mjs 会检查 hook 是否被 bridge 调用

6.2 feature → bridge

feature 通过注入的 bridge 使用能力。

要求：

- 只用公开方法/字段
- 不读 bridge.#私有、不 import 内核
- 写操作过 writeGate
- 需要新能力先走 SDK / services，不直接改 bridge

6.3 合同测试

test/bridge-api-contract.test.mjs 会扫描：

```
features/**/*.js
lib/message-pipeline.js
lib/reply-router.js
```

检查所有 bridge.* 引用是否有：

- 公开方法，或
- 明确允许的状态字段

新增 feature 调用新方法时，必须同步更新该测试的白名单/字段声明。

---

7. 数据、安全与观测

7.1 数据目录

```
D:\DSH\qqbot\data\        配置/状态/索引
D:\DSH\qqbot\logs\        日志
D:\DSH\qqbot\backups\     快照
D:\DSH\sing\library\      曲库
D:\DSH\qqbot\douyin\      抖音素材
```

规则：

- 数据路径写进 dataPaths
- 不把运行数据写进源码目录
- 不把密钥写进 dataPaths 的公开文件

- 大文件/模型放独立目录，不进 git/包

7.2 日志

- 正常路径用 debugLog
- 关键决策写 trace（bridge 已有）
- 不记录完整 token/cookie/密码
- 用户可见错误与内部错误分开

7.3 安全

- 权限判断分两段：工具注册期 + 出站执行期
- 注入/回放回合不写 QQ（injectDryRun 语义）
- 可疑好友申请只能同意，不能假装拒绝
- 路径访问必须 jailed，不允许 .. 逃逸
- 外部输入必须限长、限类型

---

8. 测试与质量门禁

8.1 必跑命令

```
# 全量 CI（audit + lint + size + 语法 + typecheck + 覆盖率 + 部署 smoke）
powershell -NoProfile -ExecutionPolicy Bypass -File D:\DSH\qqbot\scripts\ci.ps1

# 快速单测
npm run test:new --prefix D:\DSH\plugins\dsh-qq-onebot-bridge

# 全量单测（含 session 套件）
npm run test:all --prefix D:\DSH\plugins\dsh-qq-onebot-bridge

# 插件审计
node D:\DSH\qqbot\scripts\audit-plugins.mjs

# 健康检查
powershell -NoProfile -ExecutionPolicy Bypass -File D:\DSH\qqbot\scripts\health.ps1
```

8.2 测试分层

层	位置	说明
纯逻辑	test/*.test.mjs	core / services，依赖注入
协议 / WS	test/onebot-server.test.mjs	本地 WebSocket
fake-agent 回路	test/agent-loop-e2e.test.mjs	不调真实模型
feature 合同	test/bridge-api-contract.test.mjs	下游调用不缺失
部署 smoke	scripts/deploy-smoke.ps1	打包/回滚

8.3 覆盖率

当前口径：

```
core+services：行 97 / 分支 82 / 函数 79
all-lib（排除 bridge）：行 80 / 分支 73 / 函数 70（当前 84.59 / 75.09 / 71.03）
bridge.js：尚未进入覆盖率
```

要求：

- 新增 service 必须带测试
- 修 bug 先写能复现的测试
- 不允许为了覆盖率写空断言
- bridge 暂不追求覆盖率，但公开合同必须有测试

8.4 类型检查

```
core+services 基线：0（全绿）
bridge 基线：0（全绿）
features 基线：0（全绿）
```

规则：

- 新代码不得增加基线
- 基线只能降
- 修完一批错误要同步下调 JSON
- 不允许 @ts-nocheck 一把盖住

8.4.1 基线状态

当前**没有类型基线债务**：

```
core+services 0 / bridge 0 / features 0
```

覆盖率已恢复到标准阈值：

```
all-lib 80 / 73 / 70，当前 84.59 / 75.09 / 71.03
```

以后若因外部原因必须临时建基线：

1. 必须登记在 docs/DEBT.md
2. 必须有验收标准
3. 只能下降，不能上升
4. 新代码不得利用例外
5. 修完后同步下调 JSON 和 CI 阈值

8.5 语法 / lint

- 所有 .js 必须 node --check
- no-undef 必须为 0
- lib/core、lib/services 的 no-unused-vars 必须为 0
- bridge.js 的 dead import 必须为 0

---

9. 变更流程

9.1 新增 feature（默认路径）

1. 建 features/<name>/
2. 写 package.json（hooks、dataPaths）
3. 写 index.js（hooks）
4. 功能放本目录；纯逻辑抽 service

5. 写测试
6. 跑 audit + test:new + ci
7. 重启，确认 [feature] loaded <name>
8. 手测真实链路
9. 更新自检记

9.2 新增 hook（需要改 bridge）

1. 先确认现有 hook 不能复用
2. 在 bridge 调用点加最小接线
3. 在 features/README.md 登记 hook
4. 在 audit-plugins.mjs 能识别的调用形式里写
5. 加合同测试
6. 跑全量 CI + 重启

9.3 修改 bridge 公开 API

1. 先跑调用图
2. 保留兼容别名
3. 迁移 feature
4. 跑 bridge-api-contract
5. 调用图为 0 后删别名
6. 更新 size/typecheck 基线

9.4 架构决策

以下必须写 ADR：

- 新增长期机制（第二套插件系统、事件总线）
- 跨层依赖变化
- 替换宿主/部署形态
- 会影响回滚方式的变更

ADR 放 docs/adr/NNNN-*.md，至少包含：背景、决策、后果、迁移草案。

---

10. 部署、回滚与健康

10.1 端口

```
13080  DSH Web
6700   QQ 反向 WS
8899   权限面板
9998   语言网关
9999   音乐网关
```

10.2 部署

```
# 快照 + 重启 + 健康检查
powershell -File D:\DSH\qqbot\scripts\deploy.ps1 -Action deploy

# 只打包，不重启
powershell -File D:\DSH\qqbot\scripts\deploy.ps1 -Action deploy -NoRestart
```

10.3 回滚

```
powershell -File D:\DSH\qqbot\scripts\deploy.ps1 -Action rollback
```

回滚包在 D:\DSH\qqbot\backups\plugin\plugin-*.zip；feature 改动另存

D:\DSH\qqbot\backups\features\features-*.zip，恢复时按目录覆盖。

10.4 健康检查

```
powershell -File D:\DSH\qqbot\scripts\health.ps1
```

判定：

- 13080 listen
- 6700 established
- 启动日志无 failed to import

- 关键 feature [feature] loaded ...
- 新错误日志为空或已知

10.5 重启纪律

- 改完代码必须重启 13080
- 只重启 web，不动 WSL/NapCat，避免 QQ 重新扫码
- 重启后必须确认 6700 Established=True
- 失败立即回滚，不带着错误继续改

---

11. 文档标准

11.1 必须更新的文档

变化	更新
新 feature	features/README.md、自检记
新 hook	features/README.md、bridge 调用点
新 bridge API	合同测试、自检记
部署/回滚变化	docs/DEPLOY.md、本规范
架构变化	ADR

11.2 文档规则

- 一个主题一个事实源，不复制两份
- 历史冻结文档要标“冻结版本”
- 行数、基线等易变数字写“以 JSON 为准”，不要散落硬编码
- 文档里的命令必须能直接复制执行
- 不把 .bak、临时思路、过期日志塞进 docs

11.3 现有文档债务

- ARCHITECTURE.md 是 2026-11-06 冻结版，保留但不要当现状

- ARCHITECTURE-OVERVIEW.md 的数字需随现状更新
- DEPLOY.md 与 DEPLOYMENT.md 有重叠，后续合并
- features/README.md 是插件入口，需与本文同步

---

12. 现有例外与债务

以下是已存在、暂不强行改造的项，新增代码不得模仿：

1. features/commands/ops.js 已拆为 469 行 + ops-subcommands.js 252 行，两个文件均低于 500 行
2. 14 个 feature 的 dataPaths 已补齐（无持久化写 []，ark 写 maa\data + maa\queue）
3. bridge.js 1,119 行（预算 1,119），仍未进入覆盖率
4. feature 已纳入全量 typecheck，当前基线 0（全绿）
5. docs/ARCHITECTURE.md 为冻结版，数字不代表现状
6. D:\DSH\qqbot 下存在大量历史备份与日志，不属于源码

处理原则：

- 不为了“消灭债务”做一次性大重构
- 新功能不再新增同类债务
- 每次触碰旧文件时顺手改善一小块
- 债务清单更新到 docs/DEBT.md

---

13. 快速模板

13.1 package.json

```
{
  "name": "@fish/feature-example",
  "version": "1.0.0",
  "description": "示例功能",
  "type": "module",
  "main": "index.js",
```

```
  "dshFeature": {
    "hooks": ["registerExampleTools"],
    "dataPaths": ["D:\\DSH\\qqbot\\data\\example.json"]
  }
}
```

13.2 index.js

```
import { registerExampleTools } from './example-tools.js'

export const hooks = {
  registerExampleTools,
}
```

13.3 工具注册

```
export function registerExampleTools({ defineTool, agentCtx, route, key, bridge }) {
  agentCtx.tools.register(defineTool({
    name: 'qq_example',
    description: '示例工具',
    parameters: {},
    async execute() {
      return { ok: true, detail: 'done' }
    },
  }))
}
```

13.4 写操作

```
const deny = bridge.writeGate(route, 'qq.write.send')
if (deny) return { ok: false, detail: deny }
```

---

14. 最终检查清单

每次合并前逐项确认：

☐ 分层依赖合法

☐ feature 只依赖 SDK / 注入能力

☐ 新 hook 已登记

☐ 公开 API 未破坏，或已保留别名

☐ bridge-api-contract 通过

☐ test:new 通过

☐ scripts/ci.ps1 通过

☐ audit-plugins.mjs 输出 issues=0

☐ 类型基线未上升

☐ 行数预算未上升（有例外必须记录）

☐ 重启后 13080 / 6700 健康

☐ 真实链路手测

☐ 自检记已更新

☐ 回滚包可用

规范的目的不是让流程变慢，而是让“删一行代码”不会变成“线上少一个功能”。

第二部分　附录

A. 当前工程基线（2026-11-10）

项目	当前值	说明
bridge.js	1,119 非空行	预算 1,119
feature	14 个	全部有 dataPaths
hook	27 个	全局唯一
工具	80 个	audit 统计
typecheck	0 / 0 / 0	core / bridge / features
test:new	173 项	快速套件
CI 测试	731 项	全量套件
覆盖率	84.59 / 75.09 / 71.03	行 / 分支 / 函数
13080	listen + established	DSH Web
6700	listen + established	QQ 反向 WS

数字会变化；以 bridge-size-budget.json、typecheck-*.json、CI 输出为准。

B. Feature 与 Hook 清单

Feature	Hooks
ark	registerArkTools
commands	handleOpsCommand, handleEngageCommand, handleHelp, handleAdminCommand, registerReminderTools, registerWorklogTools
contacts	registerIdentityTools, registerOwnerTools, registerQueryTools
douyin	registerDouyinTools
draw	registerFeature
events	handlePrivateFiles, handleIncomingVideo, handleGroupFiles, onNotice
jobs	pollAnalyzeResults

media	registerMediaTools
memory	registerMemoryTools
pvz	registerPvzTools
self	registerSelfTools
sing	registerSingTools
usage	registerUsageTools
voice	handleVoiceReading, sendTts, synthVoiceFile, handleVoice

新 hook 必须同时更新 features/README.md 与本表；以 status.json / audit-plugins.mjs 为准。

C. 命令速查

目的	命令
全量 CI	powershell -NoProfile -ExecutionPolicy Bypass -File D:\DSH\qqbot\scripts\ci.ps1
快速单测	npm run test:new --prefix D:\DSH\plugins\dsh-qq-onebot-bridge
全量单测	npm run test:all --prefix D:\DSH\plugins\dsh-qq-onebot-bridge
插件审计	node D:\DSH\qqbot\scripts\audit-plugins.mjs
类型检查	powershell -File D:\DSH\qqbot\scripts\typecheck.ps1
健康检查	powershell -File D:\DSH\qqbot\scripts\health.ps1
部署	powershell -File D:\DSH\qqbot\scripts\deploy.ps1 -Action deploy
只打包不重启	powershell -File D:\DSH\qqbot\scripts\deploy.ps1 -Action deploy -NoRestart
回滚	powershell -File D:\DSH\qqbot\scripts\deploy.ps1 -Action rollback
插件状态	powershell -File D:\DSH\qqbot\scripts\features.ps1

	status

D. 新 Feature 实操模板

D.1 目录

```
features/example/
  package.json
  index.js
  example-tools.js
  README.md          # 推荐
```

D.2 package.json

```
{
  "name": "@fish/feature-example",
  "version": "1.0.0",
  "description": "示例功能",
  "type": "module",
  "main": "index.js",
  "dshFeature": {
    "hooks": ["registerExampleTools"],
    "dataPaths": ["D:\\DSH\\qqbot\\data\\example.json"]
  }
}
```

D.3 index.js

```
import { registerExampleTools } from './example-tools.js'

export const hooks = {
  registerExampleTools,
}
```

D.4 工具实现

```
import { existsSync, readFileSync } from 'node:fs'

export function registerExampleTools({ defineTool, agentCtx, route, key, bridge }) {
  agentCtx.tools.register(defineTool({
    name: 'qq_example',
```

```
    description: '示例工具',
    parameters: {},
    async execute() {
      const deny = bridge.writeGate(route, 'qq.write.send')
      if (deny) return { ok: false, detail: deny }
      return { ok: true, detail: 'done' }
    },
  }))
}
```

D.5 最小测试

```
import test from 'node:test'
import assert from 'node:assert/strict'
import { registerExampleTools } from '../../features/example/example-tools.js'

test('注册示例工具', () => {
  const tools = []
  registerExampleTools({
    defineTool: (def) => def,
    agentCtx: { tools: { register: (t) => tools.push(t) } },
    route: {},
    key: 'u:1',
    bridge: {},
  })
  assert.equal(tools.length, 1)
})
```

D.6 完成检查

```
[ ] package/index 一致
[ ] 只 import SDK
[ ] dataPaths 声明
[ ] 工具名唯一
[ ] 写操作过 gate
[ ] 有测试
[ ] audit issues=0
[ ] test:new 通过
[ ] CI OK
[ ] [feature] loaded
[ ] 手测真实链路
```

E. 代码审查清单

E.1 分层与接口

☐ feature 没有 import 内核内部文件

☐ feature 没有 import 另一个 feature

☐ 新能力通过 SDK 或注入的 bridge 获得

☐ bridge 只增加接线，没有领域分支

☐ 公开 API 没有无兼容的删除/改名

E.2 数据与安全

☐ dataPaths 覆盖所有持久化路径

☐ 写文件是原子的

☐ 路径没有 .. 逃逸

☐ 外部输入限长/限类型

☐ 日志没有 token / cookie / 密码

☐ 删除可恢复

E.3 错误与文案

☐ 用户看到的是可读原因，不是堆栈

☐ 关闭分支有真实原因

☐ 异常有 debugLog / trace

☐ 异步有超时

☐ 失败不会卡死主流程

E.4 测试与运维

☐ 新增逻辑有单测

☐ 契约测试过

☐ test:new 过

☐ ci.ps1 过

☐ 重启后 13080 / 6700 健康

☐ 快照 / 回滚可用

☐ 自检记已更新

F. 事故复盘与防线

F.1 `rememberText is not a function`

- 现象：每条入站消息都回“处理失败”
- 根因：createRecentRuntime 内部定义了 rememberText，return 对象漏掉
- 修复：补 return + 回归测试
- 防线：工厂返回对象必须有公开方法测试

F.2 `bridge.engageOn is not a function`

- 现象：互动命令在 QQ 里失败
- 根因：为了压行删掉了仍被 feature 调用的公开方法
- 修复：恢复方法 + bridge-api-contract 测试
- 防线：删除公开 API 前必须跑下游调用图

F.3 行数口径不一致

- 现象：测试说 1119，CI 说 1480
- 根因：一个统计非空行，一个统计总行
- 修复：统一非空行
- 防线：脚本与测试共用同一口径

F.4 不可达基线

- 现象：CI 永远红
- 根因：基线写成 0 / 70，但实际是 48 / 67
- 修复：先建真实基线，再只降不升，最后清零
- 防线：基线必须是当前真实值，不能写愿望

G. 术语表

术语	含义
feature	features/<name>/ 下的功能插件
hook	feature 暴露给 bridge 的命名入口
SDK	lib/sdk/index.js，feature 唯一允许 import 的内核内容
bridge API	注入给 feature 的公开能力，bridge.*
gate	出站写操作闸门，负责权限/干跑/限流
dry-run	只记录不真正发 QQ
inject / replay	注入/回放回合，不写 QQ
dataPaths	feature 声明使用的持久化路径
trace	结构化决策记录，排障用
contract test	检查下游调用是否仍有对应公开成员

H. 修订记录

版本	日期	说明
1.0	2026-11-10	初版全量工程标准
1.1	2026-11-10	14 feature / 27 hooks 全量 typecheck 清零；dataPaths 补齐；ops.js 拆分；覆盖率恢复 70；补充规范书与附录

规范的目的不是让流程变慢，而是让“删一行代码”不会变成“线上少一个功能”。